Document number	P3259R0
Date	2024-05-09
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

const by default

Table of contents

Abstract

Pure reference types with shallow `const` semantics would benefit from allowing adding a `const` modifier to a class definition in order to make them `const` by default and hence more consistent with the behavior of lvalue references.

Motivational Examples

function_ref

given

```
template<class... S> class function_ref;    // not defined

template<class R, class... ArgTypes>
class function_ref<R(ArgTypes...) cv noexcept(noex)> const// const by default
{
  // ...
};
```

usage

```
void action(){}

void some_other_action()
{
  function_ref<void ()> fr1{ action };// const by default
  mutable function_ref<void ()> fr2{ action };// const has been disengaged
  fr1 = function_ref<void ()>{ action };// ill formed
  fr2 = function_ref<void ()>{ action };// well formed
  std::vector<function_ref<void ()>> v1{ fr1, fr2 };// compiler error
  std::vector<mutable function_ref<void ()>> v2{ fr1, fr2 };// ok
}
```

optional

given

```
template <class T>
class optional<T&> const// const by
{
  // ...
};
```

usage

```
int i = 42;
optional<int&> oi1{ i };// const by default
optional<const int&> oi2{ i };// const by default
mutable optional<int&> oi3{ i };// const has been disengaged
mutable optional<const int&> oi4{ i };// const has been disengaged
oi1 = i;// ill formed
oi2 = i;// ill formed
oi3 = i;// well formed
oi4 = i;// well formed
std::vector<optional<const int&>> v1{ oi1, oi2, oi3, oi4 };// compiler error
std::vector<mutable optional<const int&>> v2{ oi1, oi2, oi3, oi4 };// ok
```

Motivation

Lvalue references are immutable. "A reference is similar to a const pointer such as int* const p (as opposed to a pointer to const such as const int* p)" [1] References and constant pointers are easier to reason about dangling. Since references are not reseatable and constant pointers are not by default reseatable, they are just aliases to the referent that they point to. This means at compile time, once they point to a global, local or temporary, they always point to said global, local or temporary. Further since the constness is baked in, a programmer is less likely to forget to add it.

Making pure reference types with shallow `const` semantics such as `function_ref` [2] and `optional<&>` [3] `const by default` would make them more consistent with lvalue references. This will make them easier to use safely as it is easier to reason about the lifetimes of the referents that they point to.

More Potential Examples

```
class a const{}; // const by default
class b mutable{}; // same as class b {};
class c const(true){}; // same as class c const {};
class d const(false){};// same as class c mutable {};
class e mutable(true){}; // same as class c mutable {};
class f mutable(false){};// same as class c const {};
```

While at a minimum, only adding `const` to a class definition is needed, the other examples would be beneficial for completeness and to ease their usage in library definitions.

Should've, Could've, Would've

While not a part of this proposal, there are places in the standard library where types are implicitly const by default. For instance, adding the const by default class modifier to `source_location`, `initializer_list`, `numeric_limits`, `integer_sequence`, `ration` and all of the type traits would not have negative consequences as all of its member access is currently `const` only. It should also be noted that `stacktrace_entry`, `span`, `mdspan`, `locale` and all of the exceptions would also have been ideal candidates if it was not for their assignment operator. This is especially true of exeptions since they are usually thrown by value, catched by reference and rarely if ever reassigned mutably. The `span` class would also have been ideal since it is a lightweight pure reference type like `function_ref` and `optional<&>`. The `basic_string_view` is also in a similar situation of `span`, if it wasn't for its assignment operator, `remove_prefix`, `remove_suffix` and `swap` member functions. The last three member functions just as easily could have created a new `basic_string_view` since it is a cheap stack type like `span`.

References

1. https://isocpp.org/wiki/faq/references#reseating-refs (https://isocpp.org/wiki/faq/references#reseating-refs) ↩️

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html) ↩️

3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r4.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r4.pdf) ↩️